

Algorithms (DAA)

Asymptotic Notation

Behavior of a function when the value approaches to ∞ .

1) Big O (O)

2) Big Omega (Ω)

3) Theta (Θ)

4) Little O (o)

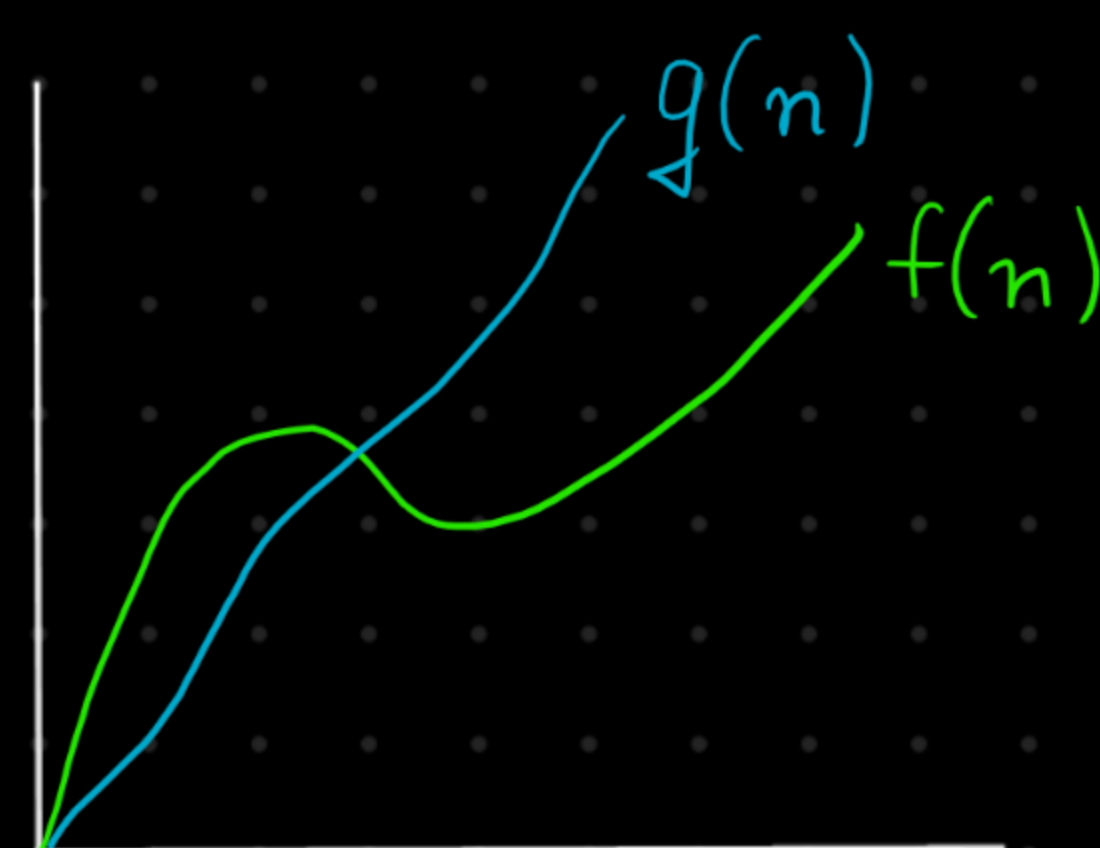
5) Little Omega (ω)

a) Big O $\rightarrow$ we have two function $f(n)$ & $g(n)$

$f(n)$ is $O(g(n))$, when

$\hookrightarrow$ order of

$$f(n) \leq C \cdot g(n) \forall n \geq K \text{ } \&\& \exists C$$



1) Big O (O) / (Least upper bound / Tightest upper bound)

$f(n) = O(g(n))$ if and only if

$$f(n) \leq C \cdot g(n) \forall n \geq K \text{ } \&\& \exists C$$

Example - 1) $n^2 + n + 1$, Find $g(n) = ??$

Option $\rightarrow$ a) n , b) $\sqrt{n^2}$, c) $\sqrt[3]{n^3}$, d) $\sqrt[4]{n^4}$, e) $\sqrt[5]{n^5}$
 $\hookrightarrow$ most accurate

$$f(n) \leq g(n)$$

$$n^2 \leq n^2$$

$$n^2 + n \leq n^2 + n^2$$

$$n^2 + n + 1 \leq n^2 + n^2 + n^2$$

$$f(n) \leq 3 \cdot n^2$$

$$f(n) \leq C \cdot g(n)$$

$$f(n) = O(n^2)$$

$$f(n) \leq g(n)$$

$$n^2 \leq n^3$$

$$n^2 + n \leq n^3 + n^3$$

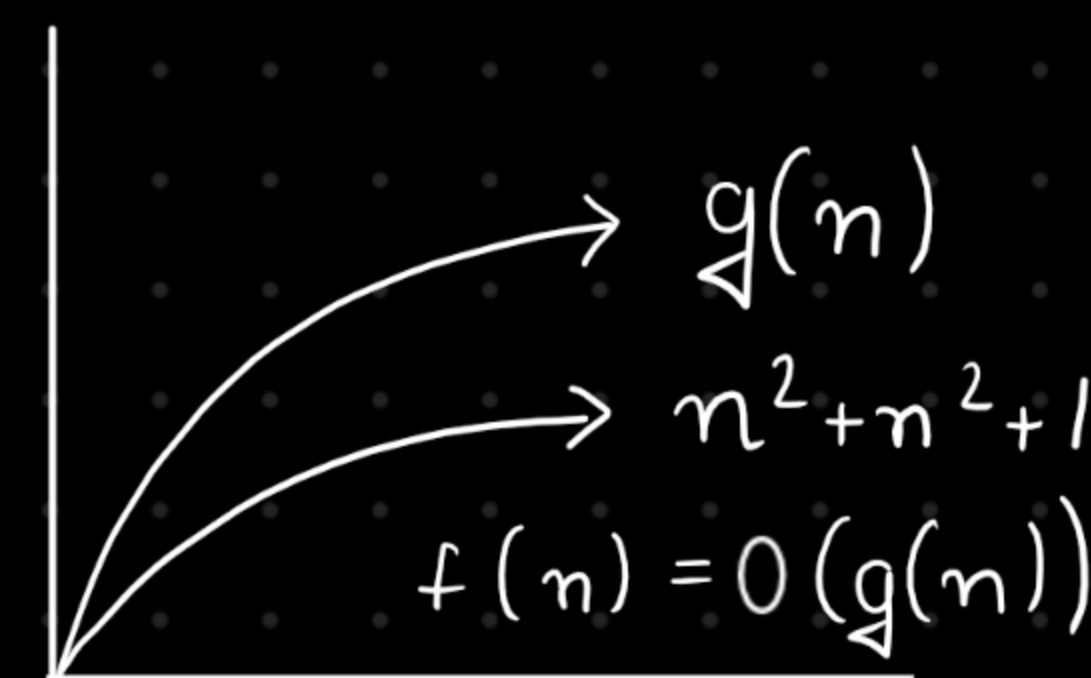
$$n^2 + n + 1 \leq n^3 + n^3 + n^3$$

$$f(n) \leq 3 \cdot n^3$$

$$f(n) \leq C \cdot g(n)$$

$$f(n) = O(n^3)$$

$$f(n) = O(n^2) = O(n^3) = O(n^4) = O(n^5)$$



2) Big Omega (Ω) (greatest lower bound / tightest lower bound)

$f(n)$ & $g(n)$ are two functions

$f(n) = \Omega(g(n))$ if and only if

$$f(n) \geq c \cdot g(n) \quad \forall n \geq k \quad \&\& \quad \exists c$$

Example - $f(n) = n^2 + n + 1$, find $g(n)$

Case - 1)

$$f(n) \geq g(n)$$

$$n^2 \geq n$$

$$n^2 + n \geq n$$

$$n^2 + n + 1 \geq n$$

$\underbrace{\hspace{1.5cm}}$

$$f(n) \geq g(n)$$

$$f(n) = \Omega(n)$$

Case - 2

$$f(n) \geq g(n)$$

$$n^2 \geq n^2$$

$$n^2 + n \geq n^2$$

$$n^2 + n + 1 \geq n^2$$

$$f(n) \geq g(n)$$

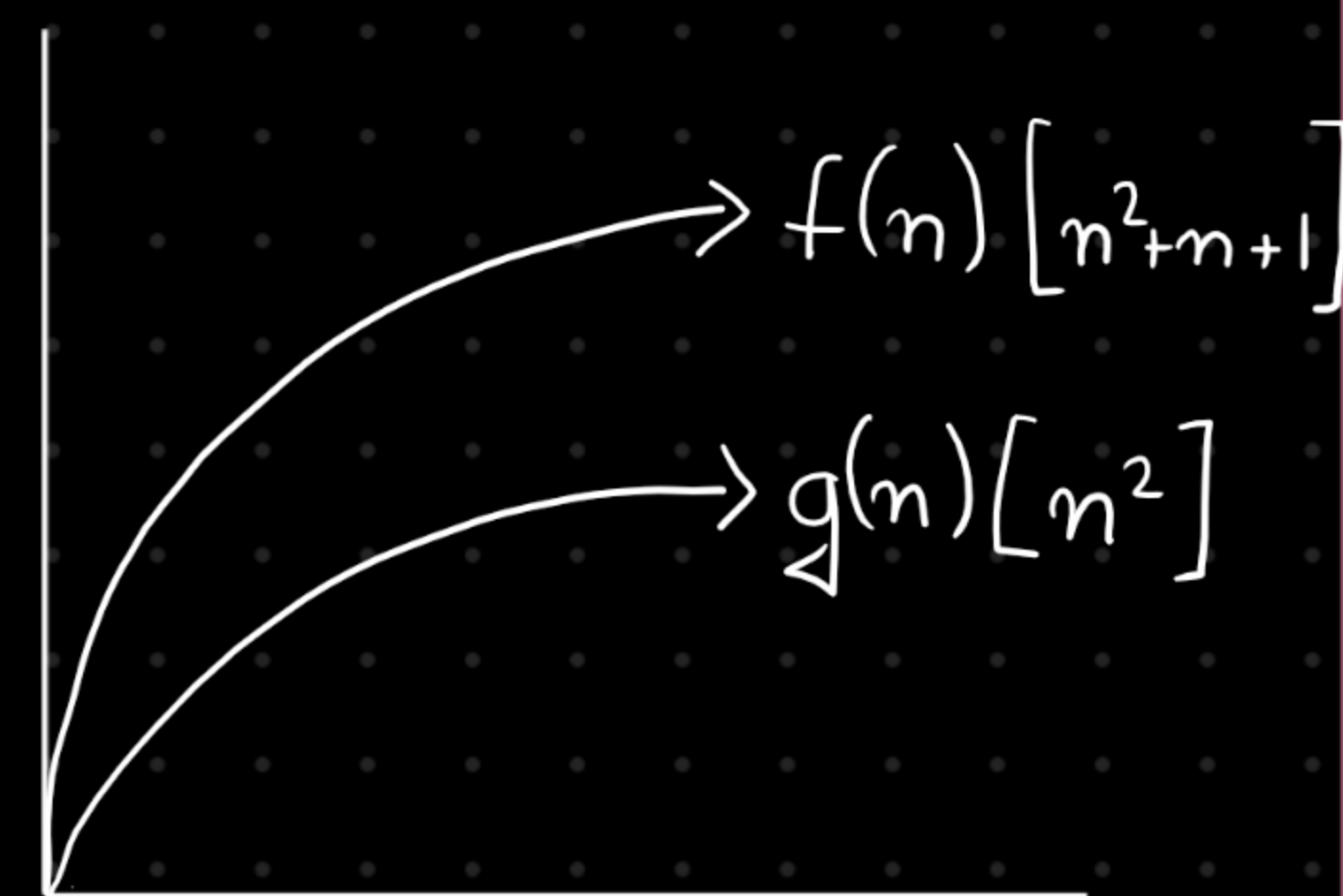
$$f(n) = \Omega(n^2)$$

Case - 3

$$f(n) \geq g(n)$$

$$n^2 \geq n^3 \rightarrow \times$$

$$\text{hence } f(n) = \Omega(n) = \Omega(n^2)$$



3) Theta (Θ)

$f(n) = \Theta(g(n))$ if & only if

$$f(n) = O(g(n))$$

&&

$$f(n) = \Omega(g(n))$$

4) Little O $\sim$

$f(n) = o(g(n))$ if and only if

$$f(n) < c \cdot g(n) \quad \forall n \geq k \quad \&\& \quad \forall c$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Example $\leadsto f(n) = n^2, g(n) = n^3$

$$\lim_{n \rightarrow \infty} \frac{n^2}{n^3} = \frac{1}{n} = \frac{1}{\infty} = 0 \quad \underline{\text{Ans}}$$

5) Little Omega (ω) $\leadsto$

$\hookleftarrow f(n) = \omega(g(n))$ if and only if $\hookrightarrow$

$$f(n) > C \cdot g(n) \quad \forall n \geq K \quad \forall C$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

Example) $f(n) = n^3, g(n) = n^2$

$$\lim_{n \rightarrow \infty} \frac{n^3}{n^2} = n = \infty \quad \underline{\text{Ans}}$$

Note

$$1) f(n) = O(g(n)) \text{ then } C \cdot f(n) = O(g(n))$$

$$2) f(n) = O(g(n)) \text{ \& } h(n) = O(e(n))$$

$$f(n) + h(n) = O(\max(O(g(n)), O(e(n))))$$

$$3) f(n) \cdot h(n) = O(f(n) \cdot h(n))$$

A priori Analysis $\rightarrow$ Order of magnitude of a statement
 $\hookrightarrow$ # times any statement is executing

Example - 1) $x = y + z \rightarrow (\text{constant}) - C$

$O(1) \rightarrow$ Constant time complexity.

Example - 2) $x = y + z - \textcircled{1} \rightarrow 1 \text{ time}$

for (int $i = 0$, $i < n$, $i++$) {

$x = y + z - \textcircled{2} \rightarrow n \text{ times}$

$i = 0 \quad i = 1 \quad \dots \quad i = n-1$
 $x = y + z \quad x = y + z \quad \dots \quad x = y + z \quad \left. \vphantom{\begin{matrix} i = 0 \\ i = 1 \end{matrix}} \right\} n \text{ times}$

$(n+1) \text{ times} \Rightarrow O(n)$

Example - 3) $x = y + z - \textcircled{1} \rightarrow 1$

for (int $i = 0$, $i < n$, $i++$) {

$x = y + z - \textcircled{2} \rightarrow n \text{ times}$

}

for (int $i = 0$; $i < n$; $i++$) {

for (int $j = 0$; $j < n$; $j++$) {

$x = y + z - \textcircled{3} \rightarrow n^2 \text{ times}$

}

Example) $n = 5$

$i = n$

while ($i > 1$):

$i--$

print("Hello") — ①

$(n - 1) \rightarrow O(n)$

Example - 5) $i = n$

while ($i > 1$) {

$i = i - 3$

print("Hi")

$n = 9$

$i = 6$

$i = 3$

$i = 6$

$i = 3$

$i = 0$

$\hookrightarrow \times$

Hi

Hi

$10/3 = 3.33$ { take upper bound }

Example 6) $i = 1$

$O(\log_2 n)$ { while ($i < n$):
 $i = 2 * i \leftrightarrow i = 3 * i$
print(i). $\hookrightarrow O(\log_3 n)$

$O(\log n)$

$n = 64$

$i < 64$

$2 < 64$

$4 < 64$

$8 < 64$

$i = 2$

$i = 4$

$i = 8$

$i = 16$

2 ✓

4 ✓

8 ✓

16 ✓

$16 < 64$

$32 < 64$

$64 \not< 64$

$i = 32$

$i = 64$

$\hookrightarrow \times$

32 ✓

64

Example - 7) $i = n$
while $i > 2$:
 $i = i^{1/2}$

$$n = 256 \rightarrow 256 > 2 \rightarrow 16 > 2 \rightarrow 4 > 2 \rightarrow 2 \not> 2$$
$$\hat{n} = 256 \quad \hat{i} = 16 \quad \hat{l} = 4 \quad \hat{c} = 2 \quad \text{LX}$$

$n^{1/2^k} = 2$ → stopping criteria

$$\Rightarrow \log_{\sqrt{2}} n^{1/2^k} = \cancel{\log_{\sqrt{2}} 2} \cdot 1 \Rightarrow \frac{1}{2^k} \log_{\sqrt{2}} n = 1 \Rightarrow \log_{\sqrt{2}} n = 2^k$$

$$\log(\log_2^n) = \log_2 2^k \Rightarrow \log(\log_2^n) = k \log_2 2^1$$

$$K = \log_4(\log_2^n) \rightarrow O(\log_4(\log_2^n))$$

Complexity class

- 1) Constant - $O(1)$
- 2) Logarithmic - $O(\log n)$
- 3) Linear - $O(n)$
- 4) Quadratic - $O(n^2)$
- 5) Cubic - $O(n^3)$
- 6) Polynomial - $O(n^c)$
- 7) Exponential - $O(c^n)$

Increasing Order time Complexity

$$\begin{array}{l} \delta) \quad n! < n^n \\ \quad 2^n < n^n \\ \quad n! > 2^n \end{array} \left. \vphantom{\begin{array}{l} \delta) \quad n! < n^n \\ \quad 2^n < n^n \\ \quad n! > 2^n \end{array}} \right\} \boxed{2^n < n! < n^n}$$

$$\begin{array}{l} \text{a) } \log n < n \\ (\log n)^2 < n \\ (\log n)^3 < n \\ \vdots \\ (\log n)^{1000} < n \end{array} \left| \begin{array}{l} \rightarrow \text{But } \boxed{(\log n)^{\log n} > n} \rightarrow \text{True} \end{array} \right.$$

Recurrence Relation

Practice sheet - IV

Q) Solve questions with substitution or recursive Tree Approach.

$$1) T(n) = 2T(n-1), T(1) = 1$$

$$\Rightarrow T(n-1) = 2T(n-1-1) = 2T(n-2)$$

$$\Rightarrow T(n) = 4T(n-2)$$

$$\Rightarrow T(n-2) = 4T(n-3)$$

$$\Rightarrow T(n) = 8T(n-3)$$

⋮
K times

$$T(n) = 2^K T(n-K), \text{ where } n-K=1$$

$$\Rightarrow K = n-1$$

$$\Rightarrow T(n) = 2^{(n-1)} T(n-n+1)$$

$$T(n) = 2^{(n-1)} T(1); T(1) = 1$$

$$T(n) = 2^{(n-1)} \Rightarrow O(2^n) \underline{Ans}$$

$$\text{ii) } T(n) = T(n/2) + n; \quad T(1) = 1$$

$$T(n) = T(n/2) + n$$

$$\Rightarrow T(n/2) = T(n/4) + \frac{n}{2}$$

$$\Rightarrow T(n) = T(n/4) + n/2 + n$$

$$\Rightarrow T(n/4) = T(n/8) + n/4$$

$$\Rightarrow T(n) = T(n/8) + n/4 + n/2 + n$$

k times

$$T(n) = T(n/2^k) + n/2^{k-1} + n/2^{k-2} + \dots + \frac{n}{2^2} + \frac{n}{2} + n$$

Now $\frac{n}{2^k} = 1 \Rightarrow n = 2^k$
 $\Rightarrow \log_2 n = k \log_2 2 \Rightarrow k = \log_2 n$

$$\Rightarrow T(n) = T\left(\frac{n}{2^{\log_2 n}}\right) + \frac{n}{2^{\log_2 n - 1}} + \frac{n}{2^{\log_2 n - 2}} + \dots + n$$

$$\Rightarrow T(1) + \frac{n}{2^{\log_2 n}} + \frac{n}{2^{\log_2 n}} + \frac{n}{2^{\log_2 n}} + \dots + n$$

$$\Rightarrow T(1) + 2 + 2^2 + 2^3 + \dots + n$$

$$\Rightarrow 2^0 + 2^1 + 2^2 + 2^3 + \dots + n$$

applying sum of GP $\leadsto a \cdot \frac{r^k - 1}{r - 1}$

$$= 1 \cdot \frac{(2^{\log_2 n} - 1)}{2 - 1} = (n - 1) \Rightarrow O(n) \underline{\text{A}}$$

iii) $T(n) = 8T(n/2) + \underline{n^2}$, $T(1) = 1 \rightarrow$ ill redo this

$$\Rightarrow 4T(n/2) + 4T(n/2) + n^2$$

Note
cost per level
can/cannot be
same in each
level

in our case

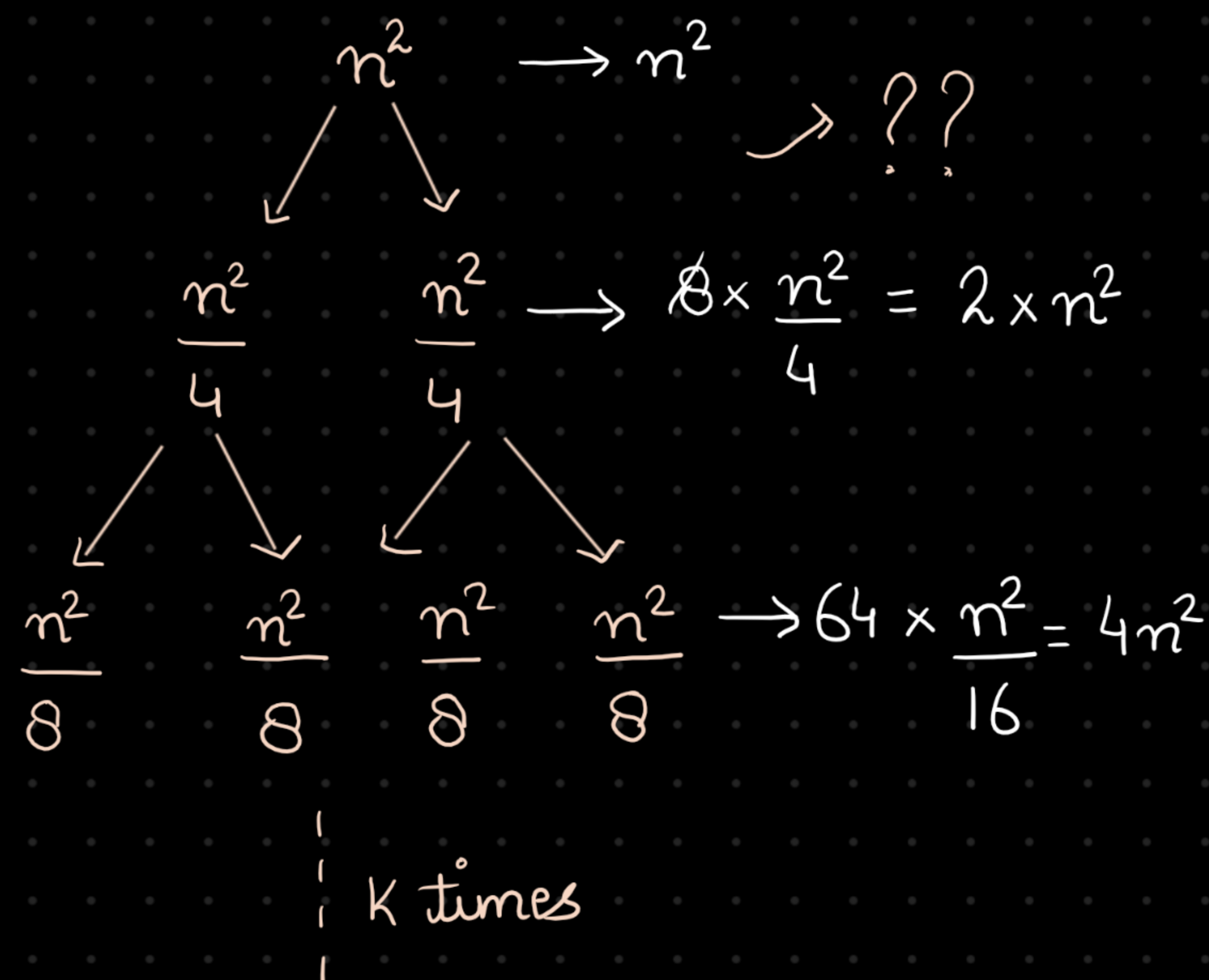
• nodes $\leadsto 8^i$

• Each node cost

$$\hookrightarrow (n/2^i)^2 = n^2/4^i$$

• Level cost

$$\hookrightarrow 8^i \times \frac{n^2}{4^i} = n^2 \cdot 2^i$$



$$\text{left side} \rightarrow \frac{n^2}{2^K} = 1$$

$\rightarrow$ same

$$\text{Right side} \rightarrow \frac{n^2}{2^K} = 1$$

$$n^2 = 2^K \Rightarrow \log_2 n^2 = K \log_2 1$$

$$K = \log_2 n^2$$

$$iii) \quad T(n) = 8 T(n/2) + n^2; T(1) = 1$$

$$\Rightarrow T(n/2) = 8 T\left(\frac{n}{4}\right) + \frac{n^2}{4}$$

$$\Rightarrow T(n) = 64 T\left(\frac{n}{4}\right) + \frac{n^2}{4} + n^2$$

$$\Rightarrow T(n/4) = 64 T\left(\frac{n}{16}\right) + \frac{n^2}{16}$$

$$\Rightarrow T(n) = 8^4 T\left(\frac{n}{16}\right) + \frac{n^2}{16} + \frac{n^2}{4} + n^2$$

|
|
|

k time

$$T(n) = 8^k T\left(\frac{n}{2^k}\right) + \frac{n^2}{2^k} + \frac{n^2}{2^{k-1}} + \dots + n^2$$

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow \log_2^n = k$$

$$T(n) = 8^{\log_2^n} T\left(\frac{n}{2^{\log_2^n}}\right) + \frac{n^2}{2^{\log_2^n-1}} + \dots + n^2$$

$$\Rightarrow 8^{\log_2^n} \cdot T(1) + 2n + 4n + \dots + n^2$$

$$\Rightarrow n^3 + 2n + 4n + 8n + \dots + n^2$$

$$\Rightarrow n^3 + \dots \text{ some gfb } \sim O(n^3) \underline{\text{Ans}}$$

$$(iv) T(n) = 2T(n/2); T(1) = 1$$

$$T(n) = 2T(n/2)$$

$$T(n/2) = 2T(n/4)$$

$$\Rightarrow T(n) = 4T(n/4)$$

$$\Rightarrow T(n/4) = 4T(n/16)$$

$$\Rightarrow T(n) = 16T(n/16)$$

⋮

k time

$$T(n) = 2^k \left(n / 2^k \right) \leadsto \frac{n}{2^k} = 1 \Rightarrow n = 2^k$$

$$\Rightarrow k = \log_2 n$$

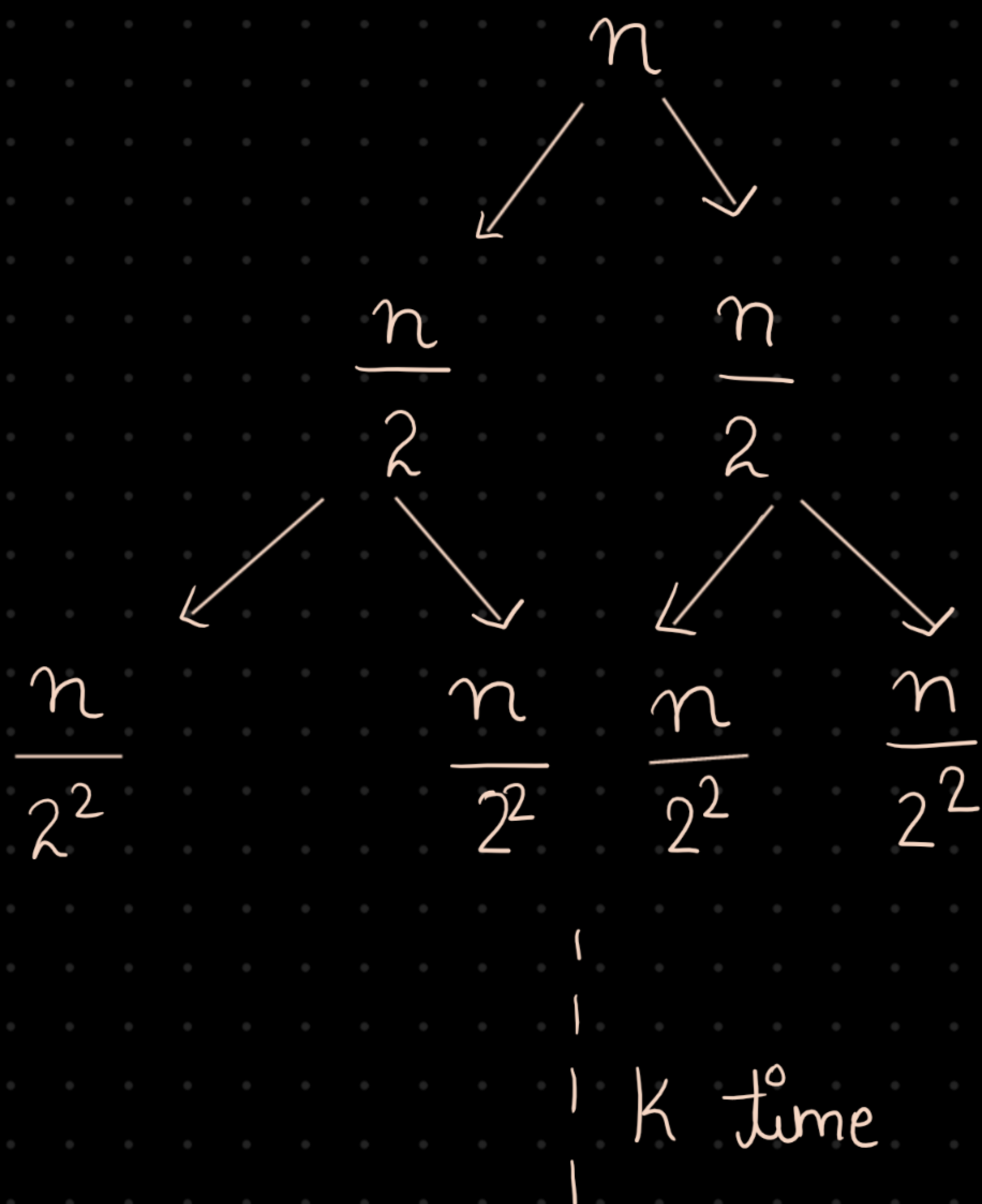
$$\Rightarrow 2^{\log_2 n} \left(n / 2^{\log_2 n} \right)$$

$$\Rightarrow n \cdot 1 = O(n) \underline{\quad}$$

$$v) T(n) = 2T(n/2) + 1, T(1) = 1$$

↳ Same as above $O(n)$

$$vi) T(n) = 2T(n/2) + n$$



Left side

$$\frac{n}{2^k} = 1$$

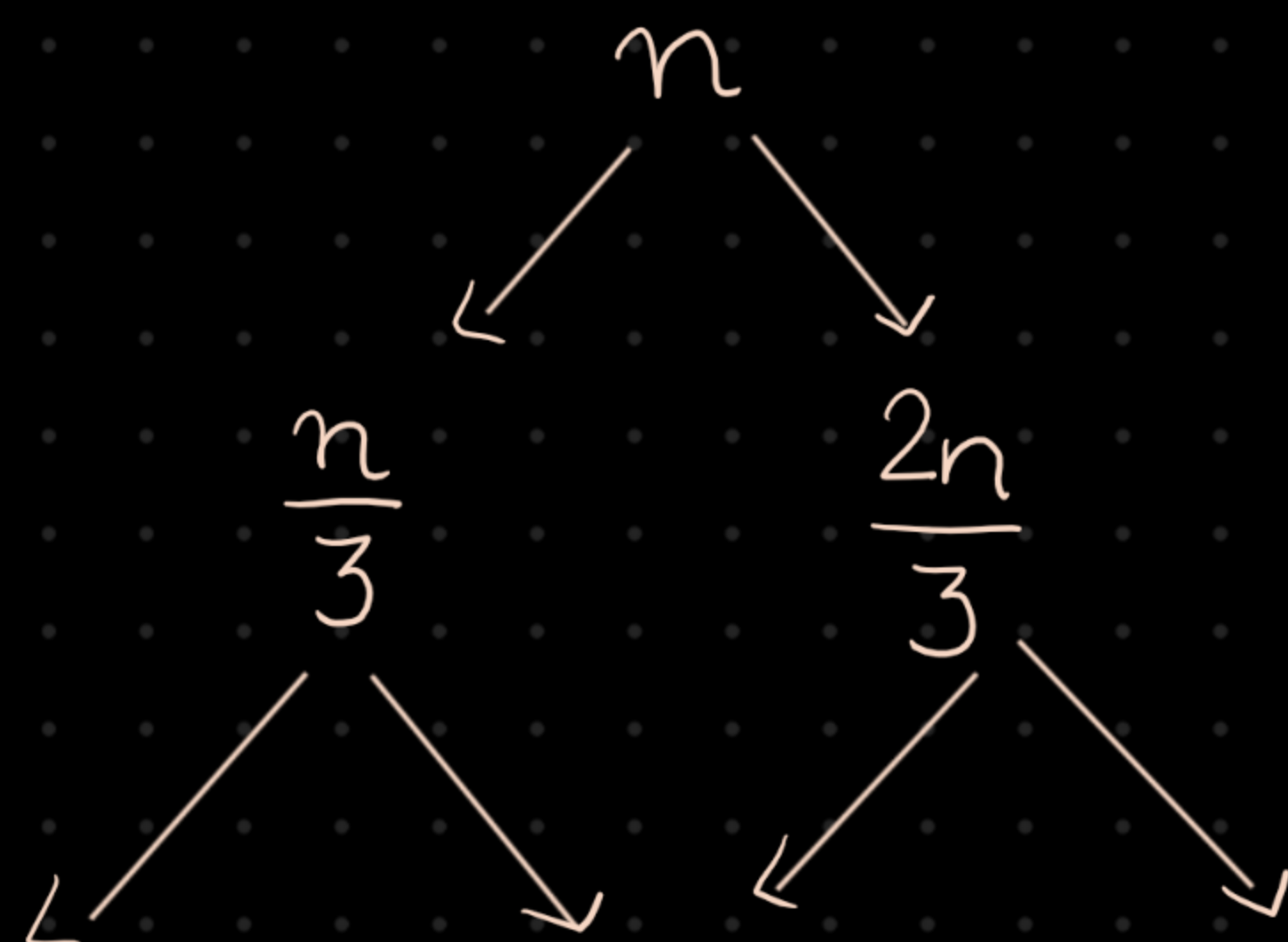
Right side

$$\frac{n}{2^k} = 1$$

$$k = \log n \text{ [In both side]}$$

$$\rightarrow O(n * k) \sim O(n \log n) \underline{\underline{A}}$$

$$vii) T(n) = T(n/3) + T(2n/3) + n, T(1) = 1$$



$$\frac{n}{3^2}$$

$$\frac{2n}{3^2}$$

$$\frac{2n}{3^2}$$

$$\frac{2^2 n}{3^2}$$

|
|
|
| k time

Left side

$$\frac{n}{3^{k_L}} = 1 \Rightarrow k = \log_{\sqrt[3]{3}} n$$

Right side

$$\frac{n}{\left(\frac{3}{2}\right)^{k_R}} = 1 \Rightarrow k = \log_{\sqrt[3]{3/2}} n$$

$$\text{since } \log_{\sqrt[3]{3/2}} n > \log_{\sqrt[3]{3}} n$$

$$\Rightarrow O(n * k) = O(n \log_{\sqrt[3]{3/2}} n) \underline{\sim}$$

$$\text{viii) } T(n) = 5 \cdot T(n/5) + C \cdot n,$$

$$T(n) = 5 \cdot T(n/5) + C \cdot n$$

$$T(n/5) = 5 \cdot T(n/5^2) + C \cdot \frac{n}{5}$$

$$T(n) = 5 \left(5 \cdot T\left(\frac{n}{5^2}\right) + C \cdot \frac{n}{5} \right) + Cn$$

$$\Rightarrow 5^2 T\left(\frac{n}{5^2}\right) + 2Cn$$

$$T(n/5^2) = 5 \cdot T\left(\frac{n}{5^3}\right) + \frac{n}{5^2}$$

$$T(n) = 5^3 \cdot T\left(\frac{n}{5^3}\right) + 3 \cdot Cn$$

k time

$$T(n) = 5^k \cdot T\left(\frac{n}{5^k}\right) + k \cdot cn$$

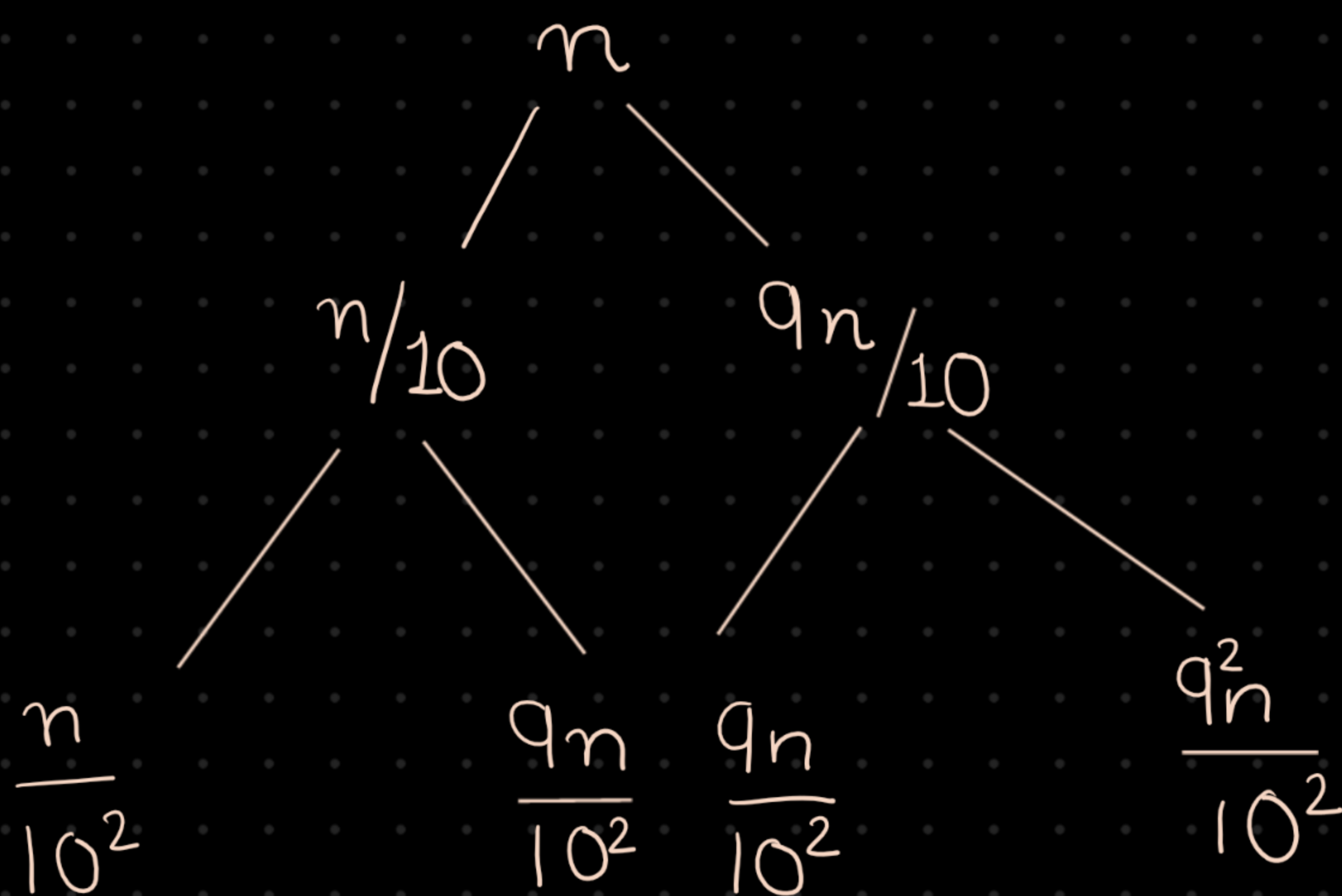
$$\frac{n}{5^k} = 1 \Rightarrow n = 5^k \Rightarrow k = \log_5 n$$

$$T(n) = 5^{\log_5 n} \cdot T\left(\frac{n}{5^{\log_5 n}}\right) + \log_5 n \cdot n$$

$$= n \cdot T(1) + n \log_5 n$$

$$= n + n \log_5 n \Rightarrow O(n \log n) \underline{\rightarrow}$$

$$9) T(n) = T(n/10) + T(9n/10) + n$$



k time

left side

$$\frac{n}{10^k} = 1$$

$$k = \log_{10} n$$

Right side

$$\frac{n}{\left(\frac{10}{q}\right)^k} = 1,$$

$$\Rightarrow k = \log_{10/q} n$$

$$\Rightarrow \log_{10/q} n > \log_{10} n$$

$$\Rightarrow O(n * k) \Rightarrow O(n * \log_{10/q} n) = O(n \log n) \underline{\underline{A}}$$